

EXAM-READY STUDY NOTES

Introduction to Artificial Intelligence

Unit C — Searching and Game Playing

A complete, structured and diagram-rich guide covering every search strategy and adversarial game-playing technique in the syllabus — built to take a first-time learner from basic concepts to a long-answer exam-ready level.

1. Searching for Solutions
2. Uninformed Search Strategies
3. Informed Search Strategies
4. Local Search Algorithms and Optimisation Problems
5. Adversarial Search
6. Search for Games (MINIMAX)
7. Alpha-Beta Pruning

Table of Contents

1. Searching for Solutions	3
2. Uninformed Search Strategies	6
BFS · DFS · Depth-Limited · Iterative Deepening · Uniform Cost Search	
3. Informed Search Strategies	13
Heuristics · Greedy Best-First Search · A* Search	
4. Local Search Algorithms and Optimisation Problems	19
Hill Climbing (all variants) · Simulated Annealing · Local Maxima, Plateaus, Ridges	
5. Adversarial Search	25
6. Search for Games — MINIMAX Algorithm	28
7. Alpha-Beta Pruning	32
Unit C Quick Revision Sheet	36
Definitions · Algorithms · Differences · Formulas · Practice Questions	

HOW TO USE THESE NOTES

Read each topic in order — concepts build on each other. Every algorithm follows the same pattern: *Definition* → *Working* → *Diagram* → *Worked Example* → *Advantages/Limitations* → *Quick Revision*. Boxes are colour-coded:

Key Concept

Formula

Example

How to Answer

.

Searching for Solutions

1.1 What is Searching in Artificial Intelligence?

Search is the general process an intelligent agent uses to find a sequence of actions that takes it from a starting situation to a desired goal, by systematically exploring the possibilities available to it. Many problems that an AI system must solve — from planning a delivery route to playing chess — can be expressed as: "Given where I am now, and the rules that let me move, find a path to where I want to be."

SIMPLE DEFINITION

Searching in AI is the process of looking through all possible sequences of actions (the **search space**) to find one particular sequence that reaches the **goal**, usually while keeping cost as low as possible.

Why Searching is Required in AI

- An intelligent agent rarely knows the direct answer to a problem; it must **reason** its way to the answer.
- Real-world problems (route planning, puzzle solving, scheduling, game playing) have too many possible situations to check by hand — search gives a systematic method to explore them.
- Search allows an agent to compare different paths and choose the best one (shortest, cheapest, fastest).
- It forms the foundation of problem-solving agents, planning systems, and game-playing programs in AI.

1.2 Problem-Solving Using Search

To make a problem "searchable", an AI system first converts the real-world problem into a **formal search problem**, defined using the components below.

Components of a Search Problem

Component	Meaning	Example (Route Finding: Ludhiana → Delhi)
Initial State	The state the agent starts in	Agent is in Ludhiana
Goal State	The state(s) the agent wants to reach	Agent is in Delhi
Actions / Operators	The set of possible moves available in any state	Drive to Chandigarh, drive to Ambala, drive to Karnal, etc.
State Space	The set of all states reachable from the initial state by any sequence of actions	All cities reachable by road from Ludhiana
Path	A sequence of states connected by actions	Ludhiana → Chandigarh → Ambala → Delhi
Path Cost	A number that measures how "expensive" a path is	Total distance in km, or total travel time
Solution	A path from the initial state to a goal state	Any valid road route reaching Delhi
Optimal Solution	The solution with the lowest path cost among all solutions	The shortest / fastest road route to Delhi

REAL-LIFE EXAMPLE — ROUTE FINDING

Imagine using a maps app to travel from Ludhiana to Delhi. The app does not know the answer directly — it treats every city as a **state**, every road as an **action** connecting two states, and then *searches* through combinations of roads until it finds a path connecting Ludhiana (initial state) to Delhi (goal state), often choosing the path with least distance or time (optimal solution).

AI EXAMPLE

In the 8-puzzle game, each arrangement of tiles is a **state**, sliding a tile is an **action**, the solved arrangement is the **goal state**, and the AI searches through possible tile movements to reach the goal — exactly the same idea used in route planning, just with a different state space.

Search Tree vs Search Graph

Search Tree	Search Graph
A tree built by an algorithm as it explores states; the same state may appear at multiple nodes if reached by different paths.	A representation of the actual state space where each state appears only once, and multiple paths between states are shown as connecting edges.
Root node = initial state; child nodes = states reachable by one action.	Nodes = states; edges = actions; can contain cycles.
Easier to implement search algorithms on.	More memory-efficient because repeated states are not duplicated.

What is a Search Strategy?

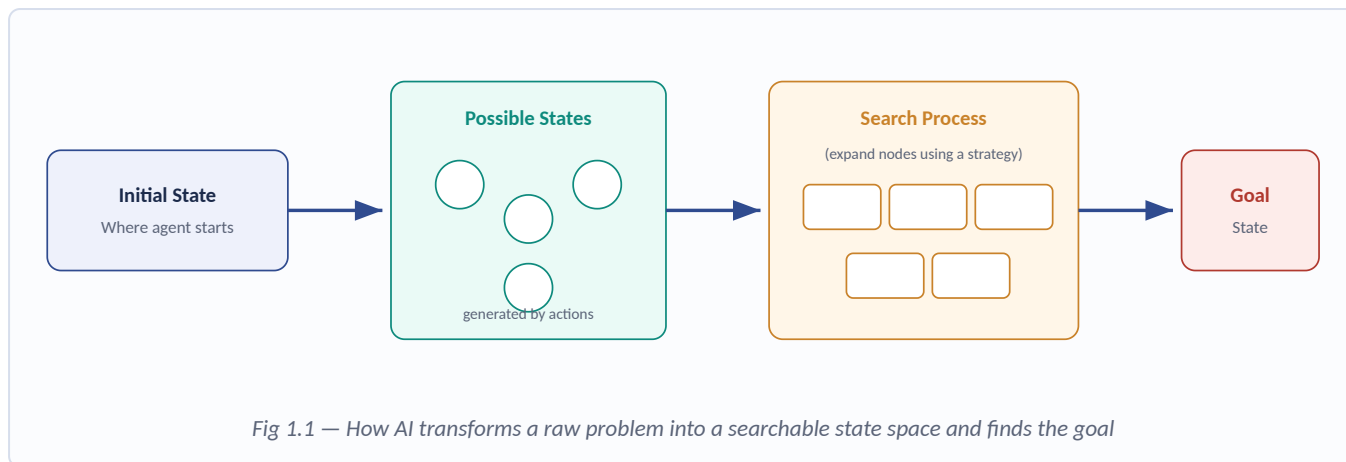
A **search strategy** is the rule an algorithm uses to decide *which node to expand next* from the set of nodes waiting to be explored (called the **frontier** or **open list**). Different strategies (BFS, DFS, A*, etc.) simply differ in this one choice — and that single difference changes their speed, memory use, and whether they find the best solution.

Evaluating a Search Strategy

Every search strategy is judged using four standard measures:

Criterion	Meaning
Completeness	Does the strategy always find a solution if one exists?
Optimality	Does the strategy always find the lowest-cost (best) solution?
Time Complexity	How long does it take (usually measured as number of nodes generated)?
Space Complexity	How much memory is needed (usually measured as maximum nodes stored at once)?

1.3 How AI Converts a Problem into a Search Problem



To convert any problem into a search problem, the AI system defines: (1) the initial state, (2) the set of legal actions and the successor states they produce, (3) a goal test to check whether a state is the goal, and (4) a path cost function. Once these four pieces exist, any general search algorithm (BFS, DFS, A*, etc.) can be applied to solve the problem — the algorithm itself does not need to know anything specific about route-finding, puzzles, or games.

HOW TO WRITE IN AN EXAM

Introduction → Definition of search → List and explain the components (initial state, goal state, actions, state space, path, solution) → Draw the Fig 1.1 style diagram → Give the route-finding example → Mention evaluation criteria (completeness, optimality, time, space) → Conclude that this formulation is what allows generic search algorithms to solve very different real-world problems.

QUICK REVISION — SEARCHING FOR SOLUTIONS

- Search = systematically exploring a state space to find a path from initial state to goal state.
- A problem is defined by: initial state, actions, transition model, goal test, path cost.
- State space = every state reachable from the initial state.
- Search tree may repeat states; search graph does not.
- Frontier = set of nodes generated but not yet expanded.
- Strategies are compared using completeness, optimality, time complexity, space complexity.
- Route-finding and the 8-puzzle are classic examples of problems formulated as search.

Uninformed Search Strategies

WHAT IS UNINFORMED (BLIND) SEARCH?

Uninformed search strategies have **no extra information** about how far a state is from the goal — they only know the problem definition (initial state, actions, goal test). They explore the state space "blindly", distinguishing states only by their position in the search tree, not by any estimate of usefulness.

All algorithms in this section will be explained using one **common example search tree** so that their exploration orders can be directly compared.

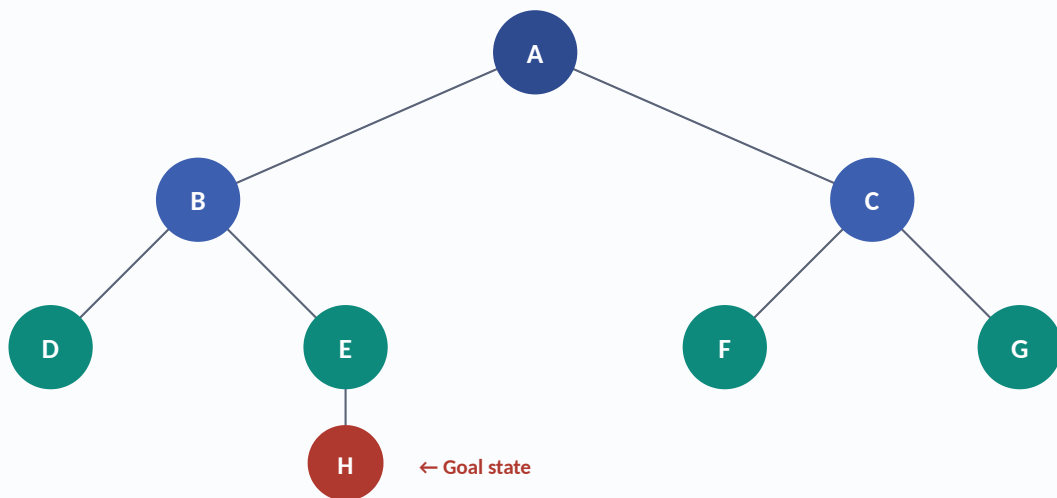


Fig 2.1 — Common Example Search Tree used for BFS, DFS, DLS and IDS (Goal = H)

2A. Breadth First Search (BFS)

Definition: BFS explores the search tree **level by level** — it expands the shallowest (closest to root) unexpanded node first, visiting all nodes at depth d before moving to depth $d+1$.

Basic Idea / Working: BFS uses a **FIFO queue** as its frontier. Nodes are added to the back of the queue when generated and removed from the front when expanded — this naturally produces level-by-level exploration.

Order of exploration on Fig 2.1: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G \rightarrow H$

PSEUDOCODE — BFS

```
function BFS(problem):
    frontier = Queue()           # FIFO
    frontier.push(initial_state)
    explored = {}
    while frontier is not empty:
        node = frontier.pop_front()
        if goal_test(node): return solution(node)
        explored.add(node)
        for child in expand(node):
            if child not in explored and child not in frontier:
                frontier.push_back(child)
    return failure
```

Property	Value	Reason
Complete	Yes	Always finds a solution if branching factor is finite
Optimal	Yes (if all step costs equal)	Finds shallowest goal first = fewest steps
Time Complexity	$O(b^d)$	b = branching factor, d = depth of shallowest goal
Space Complexity	$O(b^d)$	Must store the entire frontier (very wide levels)

ADVANTAGES

- Guaranteed to find a solution if one exists
- Finds the shortest path (fewest edges) to the goal
- Simple to implement using a queue

LIMITATIONS

- Very high memory usage — impractical for large state spaces
- Slow if the goal is deep in the tree
- Not optimal when step costs differ

2B. Depth First Search (DFS)

Definition: DFS explores as **deep as possible** along one branch before backtracking to try another branch.

Basic Idea / Working: DFS uses a **LIFO stack** (or recursion) as its frontier — the most recently generated node is expanded first, so the algorithm dives to the bottom of a branch before backtracking.

Order of exploration on Fig 2.1 (left child first): $A \rightarrow B \rightarrow D \rightarrow E \rightarrow H$ (goal found here)

PSEUDOCODE — DFS

```
function DFS(problem):
    frontier = Stack()           # LIFO
    frontier.push(initial_state)
    explored = {}
    while frontier is not empty:
        node = frontier.pop()
        if goal_test(node): return solution(node)
        explored.add(node)
        for child in expand(node):
            if child not in explored:
                frontier.push(child)
    return failure
```

Property	Value	Reason
Complete	No (Yes, only in finite state spaces without cycles)	Can loop forever down an infinite/very deep branch
Optimal	No	May find a longer path before a shorter one
Time Complexity	$O(b^m)$	m = maximum depth of the tree (can be $> d$)
Space Complexity	$O(bm)$	Only needs to store a single path plus siblings

ADVANTAGES

- Very low memory requirement compared to BFS
- Can find a solution quickly without exploring the whole tree, if lucky

LIMITATIONS

- Not complete in infinite or very deep spaces (can get stuck)
- Not optimal — may return a poor-quality solution
- Can waste time exploring an unproductive deep branch

KEY TERMS EXPLAINED SIMPLY

Complete = algorithm is guaranteed to find a solution if one exists. **Optimal** = algorithm always finds the best (lowest-cost) solution. **Time Complexity** = roughly how many nodes the algorithm must generate/check. **Space Complexity** = the maximum number of nodes the algorithm must hold in memory at once.

2C. Depth-Limited Search (DLS)

Definition: DLS is simply DFS with a predetermined **depth limit L** — nodes at depth L are treated as if they have no successors, so the search never goes deeper than L .

Why needed: It solves DFS's problem of running forever down an infinite branch by cutting the search off.

Working on Fig 2.1 with $L = 2$: Search proceeds as DFS, but any node at depth 3 (like H) is not expanded further — so H would not be reached if $L = 2$, since H is at depth 3.

Property	Value	Reason
Complete	No	Fails if the goal lies deeper than the chosen limit L
Optimal	No	Same reason as DFS
Time Complexity	$O(b^L)$	L = the fixed depth limit
Space Complexity	$O(bL)$	Same style as DFS, bounded by L

ADVANTAGES

- Prevents infinite loops down very deep/infinite branches
- Uses little memory, like DFS

LIMITATIONS

- If the correct limit L is unknown, the goal may be missed entirely
- Choosing L too small misses the goal; too large wastes time/memory

2D. Iterative Deepening Search (IDS)

Definition: IDS repeatedly runs Depth-Limited Search with an increasing depth limit ($L = 0, 1, 2, 3, \dots$) until a goal is found — combining DFS's low memory use with BFS's completeness and optimality.

Working on Fig 2.1: Run DLS with $L=0$ (just A), then $L=1$ (A,B,C), then $L=2$ (A,B,C,D,E,F,G), then $L=3$ — where H is finally found. Each iteration re-does the earlier work, but this repetition is cheap since lower levels have far fewer nodes than the deepest level.

PSEUDOCODE — IDS

```
function IDS(problem):
    for L = 0 to infinity:
        result = Depth_Limited_Search(problem, L)
        if result != cutoff: return result
```

Property	Value	Reason
Complete	Yes	Eventually reaches any finite depth
Optimal	Yes (if step costs equal)	Finds shallowest goal, like BFS
Time Complexity	$O(b^d)$	Repeated shallow work is small compared to the deepest level
Space Complexity	$O(bd)$	Only one depth-limited path is stored at a time

ADVANTAGES

- Combines BFS's completeness/optimality with DFS's low memory use
- Does not require knowing the depth of the goal in advance
- Preferred uninformed strategy when search space is large and depth is unknown

LIMITATIONS

- Re-explores upper levels of the tree multiple times (some wasted work)
- Slightly slower in practice than a single well-tuned BFS/DFS run

2E. Uniform Cost Search (UCS)

Definition: UCS expands the node with the **lowest path cost $g(n)$** so far, rather than the shallowest node — making it BFS's cost-aware cousin. It uses a **priority queue** ordered by path cost.

Working: At every step, UCS picks the frontier node with the smallest cumulative cost from the start, expands it, and updates costs of its children. It stops when the node popped from the priority queue is the goal.

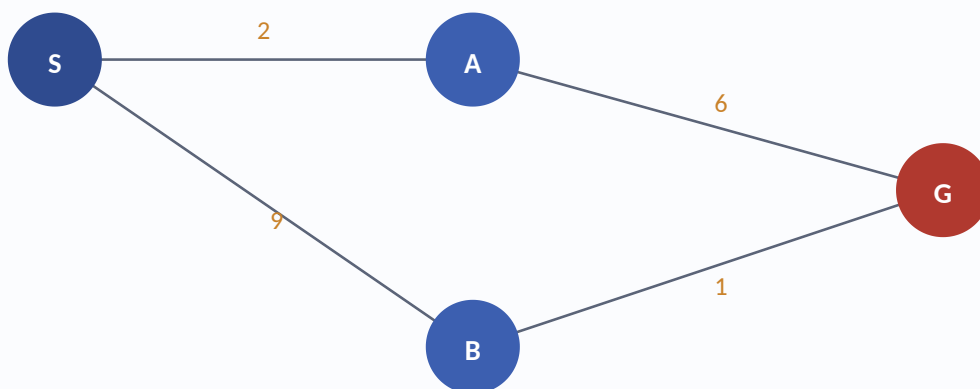


Fig 2.2 — UCS example: path $S \rightarrow A \rightarrow G$ costs $2+6=8$, path $S \rightarrow B \rightarrow G$ costs $9+1=10$. UCS correctly picks $S \rightarrow A \rightarrow G$ (cost 8), the cheaper path, even though it visits more edges than $S \rightarrow B \rightarrow G$'s single hop pattern would suggest.

PSEUDOCODE — UNIFORM COST SEARCH

```
function UCS(problem):
    frontier = PriorityQueue(ordered by g(n))
    frontier.push(initial_state, cost=0)
    explored = {}
    while frontier is not empty:
        node = frontier.pop_lowest_cost()
        if goal_test(node): return solution(node)
        explored.add(node)
        for child in expand(node):
            new_cost = g(node) + step_cost(node, child)
            if child not in explored and child not in frontier:
                frontier.push(child, new_cost)
            elif child in frontier with higher cost:
                frontier.replace(child, new_cost)
```

Property	Value	Reason
Complete	Yes	Provided step costs are $\geq$ a small positive value
Optimal	Yes	Always expands the cheapest available path first
Time Complexity	$O(b^{1+\lceil C^*/\epsilon \rceil})$	C^* = cost of optimal solution, ϵ = smallest step cost
Space Complexity	Same order as time complexity	Must store all generated nodes in the priority queue

ADVANTAGES

- Optimal even when step costs are different (unlike plain BFS)
- Complete for positive step costs

LIMITATIONS

- Can explore very large areas if step costs are small and uniform
- No knowledge of goal direction — can waste effort exploring cheap but useless paths

2.6 Comparison Table — Uninformed Search Strategies

Strategy	Complete?	Optimal?	Time	Space	Frontier Structure
BFS	Yes	Yes*	$O(b^d)$	$O(b^d)$	FIFO Queue
DFS	No	No	$O(b^m)$	$O(bm)$	LIFO Stack
Depth-Limited	No	No	$O(b^L)$	$O(bL)$	LIFO Stack + limit L
Iterative Deepening	Yes	Yes*	$O(b^d)$	$O(bd)$	Repeated DLS
Uniform Cost	Yes	Yes	$O(b^{1+\lceil C^*/\epsilon \rceil})$	same	Priority Queue (by cost)

Optimal only when all step costs are equal. b = branching factor, d = depth of shallowest goal, m = maximum depth, L = depth limit, C^ = optimal path cost, ϵ = minimum step cost.

BFS VS DFS — QUICK COMPARISON

Point	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO)
Exploration style	Level by level (wide)	Branch by branch (deep)
Memory	High	Low
Guaranteed shortest path?	Yes	No
Best used when	Goal is shallow / shortest path needed	Memory is limited / any solution is acceptable

QUICK REVISION — UNINFORMED SEARCH

- Uninformed search has no knowledge of goal distance — explores blindly.
- BFS = queue, level-by-level, complete & optimal (equal costs), but memory-heavy.
- DFS = stack, goes deep first, memory-efficient, but not complete/optimal.
- Depth-Limited Search = DFS + a fixed cut-off depth L.
- Iterative Deepening = repeated DLS with increasing L; gets BFS's guarantees with DFS's memory use.
- Uniform Cost Search = expands lowest path-cost node first using a priority queue; optimal for varying costs.

Informed Search Strategies

WHAT IS INFORMED (HEURISTIC) SEARCH?

Informed search strategies use **extra problem-specific knowledge**, beyond the problem definition, to guide the search more efficiently toward the goal. This extra knowledge is called a **heuristic** — an "educated guess" about how close a state is to the goal.

Informed vs Uninformed Search

Point	Uninformed Search	Informed Search
Knowledge used	Only the problem definition	Problem definition + heuristic function $h(n)$
Direction of search	Blind, explores in a fixed pattern	Guided toward the goal
Efficiency	Usually slower, explores more nodes	Usually faster, explores fewer nodes
Examples	BFS, DFS, UCS	Greedy Best-First Search, A*

What is a Heuristic Function $h(n)$?

A **heuristic function $h(n)$** estimates the cost of the cheapest path from node n to the goal. It is a rule-of-thumb, not a guaranteed exact value — it simply gives the search a sense of direction.

REAL-LIFE / AI EXAMPLE

In route finding, **straight-line (Euclidean) distance** to the destination city is a common heuristic — it is easy to calculate and gives a reasonable estimate of remaining distance, even though the real road distance is usually a little more.

3A. Greedy Best-First Search

Definition: Greedy Best-First Search expands the node that appears **closest to the goal**, using only the heuristic value — it always picks the node with the lowest $h(n)$, completely ignoring the cost already spent reaching that node.

Working: Nodes are stored in a priority queue ordered purely by $h(n)$. At every step, the node with the smallest $h(n)$ is expanded next.

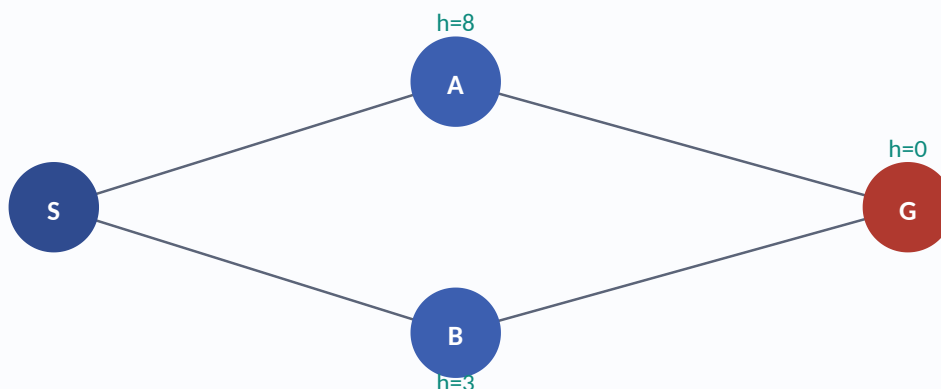


Fig 3.1 — Greedy Best-First picks B over A ($h=3 < h=8$), reaching G directly, ignoring path cost $g(n)$

Property	Value	Reason
Complete	No (Yes in finite spaces with repeated-state checking)	Can get stuck going down a misleading path
Optimal	No	Ignores actual cost $g(n)$ so far, only looks at $h(n)$
Time Complexity	$O(b^m)$ worst case	Depends heavily on heuristic quality
Space Complexity	$O(b^m)$ worst case	Must keep all generated nodes in memory

ADVANTAGES

- Usually much faster than uninformed search when the heuristic is good
- Simple to implement

LIMITATIONS

- Not complete or optimal in general
- Can be misled by a poor heuristic into long/expensive paths

3B. A* Search

Definition: A* Search combines the strengths of Uniform Cost Search and Greedy Best-First Search — it considers **both** the cost already spent reaching a node **and** the estimated cost still remaining, choosing the node that minimises their sum.

A* EVALUATION FUNCTION

$$f(n) = g(n) + h(n)$$

- $g(n)$ — the actual cost of the path from the start node to node n (known exactly)
- $h(n)$ — the heuristic estimate of the cost from node n to the goal (estimated)
- $f(n)$ — the estimated total cost of the cheapest solution path that passes through n

Admissible Heuristic

A heuristic $h(n)$ is called **admissible** if it **never overestimates** the true cost to reach the goal — in simple words, it is always "optimistic" or exactly correct, never pessimistic. This property is what guarantees A* finds the optimal solution: an admissible heuristic never causes A* to wrongly dismiss a path that is actually cheaper.

Numerical Example — How A* Chooses the Next Node

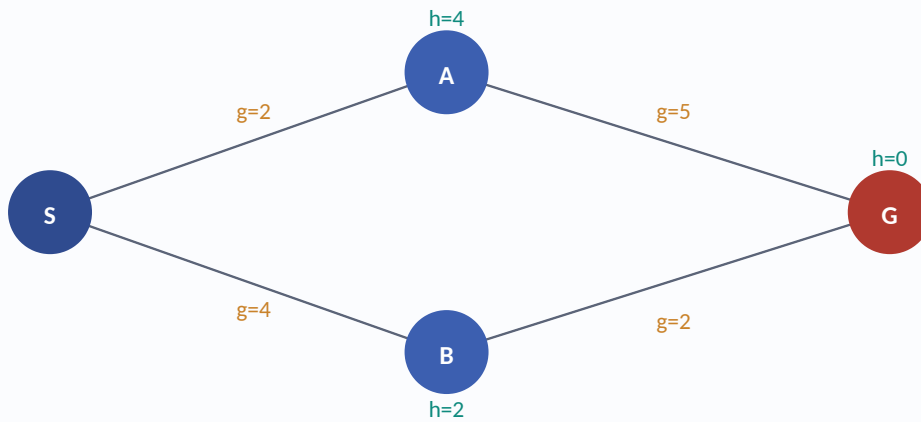


Fig 3.2 — Two paths from S to G, edge labels = step cost, node labels = heuristic $h(n)$

Step-by-step $f(n)$ calculation for Fig 3.2

Node	$g(n)$ — cost so far	$h(n)$ — estimate to goal	$f(n) = g(n) + h(n)$
A (via $S \rightarrow A$)	2	4	6
B (via $S \rightarrow B$)	4	2	6
G (via $S \rightarrow A \rightarrow G$)	$2+5=7$	0	7
G (via $S \rightarrow B \rightarrow G$)	$4+2=6$	0	6

Both A and B are tied at $f=6$ initially, so A^* may expand either first; expanding whichever leads to G with the lowest total $f(n)$ — here $S \rightarrow B \rightarrow G$, with total cost 6, is cheaper than $S \rightarrow A \rightarrow G$ (cost 7). A^* correctly identifies $S \rightarrow B \rightarrow G$ as the optimal path, because at every step it compares full path estimates, not just the heuristic.

PSEUDOCODE — A^* SEARCH

```
function A_Star(problem):
    frontier = PriorityQueue(ordered by  $f(n) = g(n) + h(n)$ )
    frontier.push(initial_state, f = h(initial_state))
    explored = {}
    while frontier is not empty:
        node = frontier.pop_lowest_f()
        if goal_test(node): return solution(node)
        explored.add(node)
        for child in expand(node):
            g_child = g(node) + step_cost(node, child)
            f_child = g_child + h(child)
            if child not in explored and child not in frontier:
                frontier.push(child, f_child)
            elif child in frontier with higher f:
                frontier.replace(child, f_child)
```

Property	Value	Reason
Complete	Yes	As long as branching factor is finite and step costs positive
Optimal	Yes	Guaranteed if the heuristic is admissible
Time Complexity	Exponential in worst case, much better with a good heuristic	Depends heavily on heuristic accuracy
Space Complexity	High — stores all generated nodes	Main practical drawback of A*

ADVANTAGES

- Complete and optimal with an admissible heuristic
- Much more efficient than uninformed search in practice
- Widely used in games, robotics, and route-planning apps

LIMITATIONS

- High memory usage — can run out of memory on very large problems
- Performance depends entirely on the quality of the heuristic used

3.3 Comparison Table — Greedy Best-First vs A*

Point	Greedy Best-First Search	A* Search
Evaluation function	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Considers cost so far?	No	Yes
Complete	No (generally)	Yes
Optimal	No	Yes (with admissible heuristic)
Speed	Fast, but can be misled	Slightly slower, but reliable

QUICK REVISION — INFORMED SEARCH

- Informed search uses a heuristic $h(n)$ to guide the search toward the goal.
- Greedy Best-First Search: $f(n) = h(n)$ only — fast but not optimal/complete in general.
- A* Search: $f(n) = g(n) + h(n)$ — combines real cost and estimated cost.
- Admissible heuristic = never overestimates the true remaining cost.
- A* is complete and optimal when its heuristic is admissible.
- Main drawback of A*: high memory usage on large problems.

Local Search Algorithms and Optimisation Problems

WHAT IS LOCAL SEARCH?

Unlike the systematic search methods seen so far (which remember the whole path from start to goal), a **local search** algorithm keeps track of only a **single current state** and tries to improve it by moving to a better **neighbouring state**. It does not care about the path taken — only about reaching a good final state. Local search is used mainly for **optimisation problems**, where the goal is to find the state that maximises or minimises some measure, rather than to reach one specific goal state.

Term	Meaning
Current State	The single state the algorithm is presently at
Neighbouring State	A state reachable from the current state by one small change/move
Objective / Evaluation Function	A function that scores how good a state is (higher = better, in a maximisation problem)

Why Local Search is Useful: It uses very little memory (only the current state, not a whole search tree) and can find reasonable solutions in huge or even infinite state spaces where systematic search is impossible — such as scheduling, circuit design, and the travelling salesman problem.

4A. Hill Climbing

Definition: Hill Climbing is a local search technique that continuously moves in the direction of **increasing value** (uphill) — at every step it moves to the neighbouring state with a better evaluation value than the current state, and stops when no neighbour is better.

PSEUDOCODE — HILL CLIMBING

```
function Hill_Climbing(problem):
    current = initial_state
    loop:
        neighbor = the highest-valued successor of current
        if value(neighbor) <= value(current):
            return current          # reached a peak
        current = neighbor
```

Real-life analogy: Imagine climbing a hill in thick fog, able to see only your immediate surroundings — at each step you simply move to whichever nearby spot is higher than where you stand, until every direction around you goes downhill.

Steepest-Ascent Hill Climbing

A variant where, instead of moving to the *first* better neighbour found, the algorithm examines **all** neighbours and moves to the one with the **highest** value — giving a more informed (but slightly more expensive) choice at every step.

Random-Restart Hill Climbing

Runs ordinary Hill Climbing multiple times, each time starting from a **randomly chosen initial state**, and keeps the best result found across all runs. This dramatically increases the chance of finding the true global maximum, since different starting points escape different local maxima.

4.2 Problems in Hill Climbing

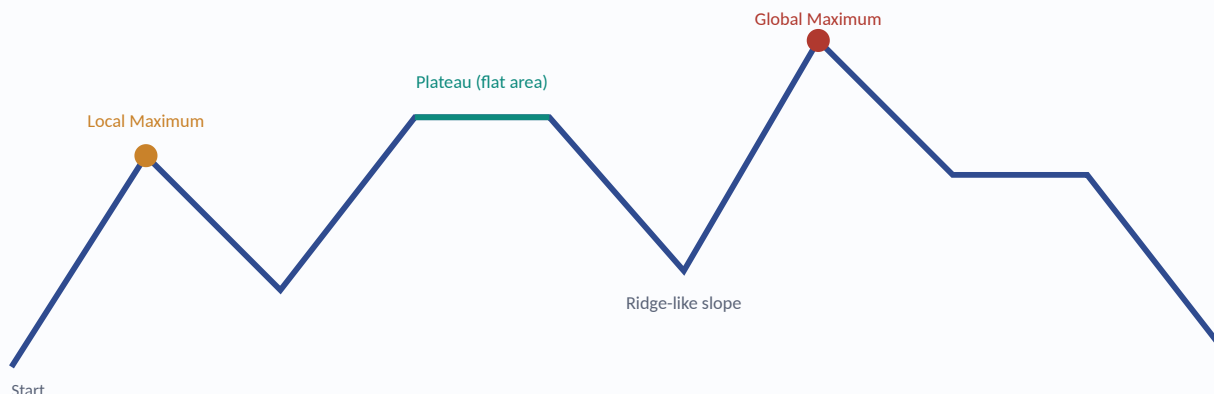


Fig 4.1 — Landscape showing Local Maximum, Global Maximum, and a Plateau along the search curve

Problem	What it means	Effect on Hill Climbing
Local Maximum	A peak that is higher than all its neighbours, but lower than the global maximum	Algorithm stops here, thinking it found the best state, and gets stuck
Global Maximum	The single highest point in the entire state space	The true goal — not always reached by plain Hill Climbing
Plateau	A flat area of the state space where neighbouring states have equal value	Algorithm cannot tell which direction is "uphill" and may wander aimlessly or stop
Ridge	A sequence of local maxima that is difficult to navigate because it requires several moves in a sequence that individually look non-improving	Simple hill climbing (one step at a time) struggles to cross it

ADVANTAGES OF HILL CLIMBING

- Very simple to implement
- Uses extremely little memory (only current state)
- Works well on problems with few local maxima

LIMITATIONS OF HILL CLIMBING

- Can get stuck at local maxima, plateaus, and ridges
- No guarantee of reaching the global maximum
- Has no memory of past states, so cannot "look back"

4B. Simulated Annealing

Basic Idea: Simulated Annealing improves on Hill Climbing by occasionally allowing moves to a **worse** state, instead of always moving uphill. This randomness lets it escape local maxima that would trap ordinary Hill Climbing.

Temperature Concept

The algorithm uses a control parameter called **temperature (T)**. T starts **high**, allowing many "bad" (downhill) moves to be accepted, and gradually **decreases** according to a cooling schedule. As T approaches zero, the algorithm behaves more and more like ordinary Hill Climbing, only accepting improving moves.

Why it can Escape Local Maxima

Because a worse move can be accepted with some probability (which depends on how much worse the move is, and the current temperature), the algorithm can move *downhill* temporarily to cross a valley and reach a taller peak elsewhere — something plain Hill Climbing can never do.

PSEUDOCODE — SIMULATED ANNEALING

```
function Simulated_Annealing(problem, schedule):
    current = initial_state
    for t = 1 to infinity:
        T = schedule(t)                # temperature at time t
        if T == 0: return current
        next = a randomly selected neighbor of current
        delta_E = value(next) - value(current)
        if delta_E > 0:
            current = next              # always accept better moves
        else:
            accept next with probability  $e^{(\text{delta\_E} / T)}$   # sometimes accept worse moves
```

REAL-LIFE ANALOGY

This mirrors **annealing in metallurgy**, where a metal is heated to a high temperature (allowing atoms to move freely and escape a poor structural arrangement) and then cooled slowly, letting atoms settle into a much stronger, more stable final structure than rapid cooling would produce.

Point	Hill Climbing	Simulated Annealing
Moves to worse states?	Never	Sometimes (probabilistically)
Risk of local maximum	High	Much lower
Extra control parameter	None	Temperature / cooling schedule
Guaranteed global optimum?	No	Yes, in theory, with a sufficiently slow cooling schedule
Speed	Fast	Slower (needs many iterations)

NOTE ON "OPTIMISATION PROBLEMS" IN LOCAL SEARCH

In this syllabus, local search is applied to **optimisation problems** — problems where the aim is to find the state that best maximises (or minimises) an objective function, rather than problems with one fixed goal state to be reached via a specific path. (This interpretation is used because the exact phrase in some syllabus copies appears with a typo/variant wording; the standard AI textbook term is "optimisation problems".)

QUICK REVISION — LOCAL SEARCH

- Local search keeps only the current state, not a full search tree — very memory-efficient.
- Hill Climbing moves to a better neighbour at every step; stops when no neighbour is better.
- Steepest-Ascent Hill Climbing checks all neighbours and picks the best one.
- Random-Restart Hill Climbing runs from many random starting points to avoid local maxima.
- Problems: Local Maximum, Global Maximum, Plateau, Ridge.
- Simulated Annealing accepts some worse moves (controlled by temperature) to escape local maxima.
- As temperature $\rightarrow 0$, Simulated Annealing behaves like plain Hill Climbing.

Adversarial Search

WHAT IS ADVERSARIAL SEARCH?

Adversarial search is the search technique used in problems where **more than one agent competes against each other**, and each agent tries to maximise its own outcome while the other tries to stop it. It is called "adversarial" because the environment includes an opponent actively working against the agent's goals — typically seen in two-player games like chess, tic-tac-toe, and checkers.

Why Adversarial Search is Required

- Normal search problems assume a single agent acting in a passive environment — but in games, an intelligent, unpredictable opponent is also choosing moves.
- The outcome does not depend on the agent's actions alone; it depends on the **interaction** between both players' choices.
- Adversarial search lets an agent plan moves that remain good **even in the worst case**, assuming the opponent always plays optimally against it.

Normal Problem Solving vs Adversarial Problem Solving

Point	Normal (Single-Agent) Search	Adversarial Search
Number of agents	One	Two (or more), competing
Environment	Passive / static	Actively opposed by an intelligent opponent
Goal	Reach a specific goal state at lowest cost	Maximise own outcome while opponent tries to minimise it
Plan quality	A single fixed sequence of actions	A strategy/policy that reacts to the opponent's moves

5.1 Key Concepts in Two-Player Games

Term	Meaning
MAX player	The player trying to maximise the outcome/utility — usually represents "our" agent
MIN player	The opponent, trying to minimise the outcome/utility for MAX
Game State	A complete description of the game at a point in time (e.g., a board position)
Game Tree	A tree where nodes are game states and edges are moves, alternating between MAX and MIN levels
Terminal State	A game state where the game has ended (win, lose, or draw)
Utility Value	A number assigned to a terminal state showing how good it is for MAX (e.g., +1 win, -1 loss, 0 draw)

SIMPLE GAME EXAMPLE

In Tic-Tac-Toe, MAX (say, playing "X") wants to reach a terminal state with utility +1 (a win for X), while MIN (playing "O") wants to reach a terminal state with utility -1 (a win for O). Every move alternates between the two players, and each is assumed to play the best move available to them at every turn.

5.2 Game Tree Diagram (MAX and MIN levels)

MAX

MIN

Terminal

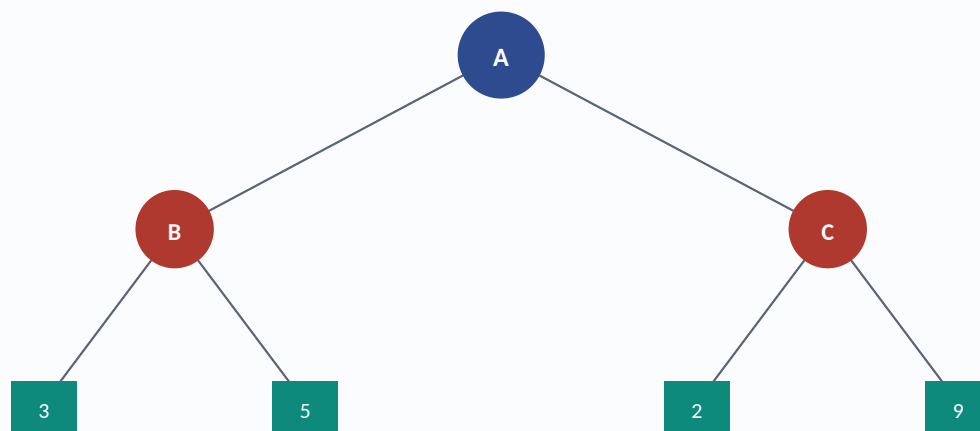


Fig 5.1 — Game tree: MAX chooses the move leading to the higher value; MIN chooses the move leading to the lower value; rectangles are terminal states with their utility values

In Fig 5.1, at node B (a MIN node) the opponent will pick the lower of 3 and 5, i.e. 3. At node C (a MIN node) the opponent will pick the lower of 2 and 9, i.e. 2. At the root A (a MAX node), our agent picks the higher of B's value (3) and C's value (2), i.e. moves toward B, guaranteeing a value of 3. This exact back-propagation process is formalised as the **MINIMAX algorithm**, covered in the next topic.

HOW TO WRITE IN AN EXAM

Define adversarial search → explain why single-agent search is insufficient for games → define MAX, MIN, game tree, terminal state, utility → draw a MAX/MIN game tree diagram → briefly explain how values propagate upward (linking forward to MINIMAX) → conclude with why this models "worst-case" optimal play against a rational opponent.

QUICK REVISION — ADVERSARIAL SEARCH

- Adversarial search = search with a competing, intelligent opponent.
- MAX wants to maximise utility; MIN wants to minimise it.
- Game tree alternates MAX and MIN levels down to terminal states.
- Utility value scores how good a terminal state is for MAX.
- The agent plans assuming the opponent always plays optimally (worst-case reasoning).

Search for Games — The MINIMAX Algorithm

6.1 Formulating a Game as a Search Problem

Component	Meaning in a Game
Initial State	The starting configuration of the game (e.g., empty board) plus whose turn it is
Players	Player(s) function returns which player (MAX/MIN) is to move in a given state
Actions	The set of legal moves available in a state
Result / Successor Function	Defines the state produced by making a move in a given state
Terminal Test	Checks whether the game has ended in a given state
Utility Function	Assigns a numeric value to terminal states (e.g., +1 win, 0 draw, -1 loss for MAX)
Optimal Move	The move that leads to the best guaranteed outcome, assuming the opponent also plays optimally

6.2 The MINIMAX Algorithm

BASIC CONCEPT

MINIMAX is an algorithm that computes the optimal move for MAX by assuming **both players play perfectly** — MAX always picks the move leading to the highest achievable value, and MIN always picks the move leading to the lowest achievable value, at every level of the game tree.

- **MAX level:** the value of a MAX node is the **maximum** of its children's values.
- **MIN level:** the value of a MIN node is the **minimum** of its children's values.
- Values are computed bottom-up, starting from terminal states, and **propagated upward** to the root.
- The best move at the root is whichever child produced the value chosen at the root.

Step-by-Step Working

1. Generate the complete game tree down to terminal states.
2. Apply the utility function to every terminal state to get its value.
3. Moving upward, at every MIN node take the minimum of the children's values.
4. At every MAX node, take the maximum of the children's values.
5. Continue until the root's value is known; the child that produced this value is the optimal first move.

6.3 Numerical Example — Full MINIMAX Calculation

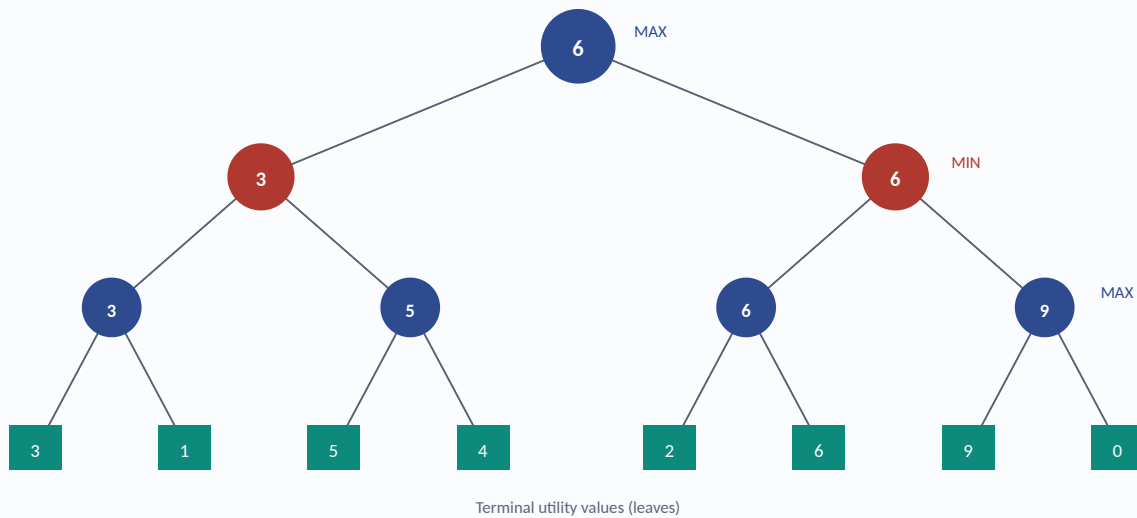


Fig 6.1 — Complete MINIMAX tree: values are computed bottom-up (leaves → MAX row → MIN row → root)

MINIMAX Calculation, Level by Level (Fig 6.1)

Level	Node	Rule	Calculation	Value
Bottom MAX level	leftmost pair	max(children)	$\max(3, 1)$	3
Bottom MAX level	2nd pair	max(children)	$\max(5, 4)$	5
Bottom MAX level	3rd pair	max(children)	$\max(2, 6)$	6
Bottom MAX level	4th pair	max(children)	$\max(9, 0)$	9
MIN level (left)	node B	min(children)	$\min(3, 5)$	3
MIN level (right)	node C	min(children)	$\min(6, 9)$	6
Root (MAX)	node A	max(children)	$\max(3, 6)$	6

Note: the leaf values above (3,1 / 5,4 / 2,6 / 9,0) refine Fig 6.1 to show a fully computable 4-level tree; working through them level by level is exactly how MINIMAX values propagate upward in any game tree, regardless of size.

HOW FINAL BEST MOVE IS SELECTED

Once the root's value is known (here, 6, coming from the right subtree C), MAX simply chooses the action that leads to the child with that value — here, the move toward node C, because it guarantees the best achievable outcome (6) assuming MIN always defends optimally afterward.

PSEUDOCODE — MINIMAX

```
function MINIMAX(state):
    if Terminal_Test(state): return Utility(state)

    if Player(state) == MAX:
        best = -infinity
        for each action in Actions(state):
            best = max(best, MINIMAX(Result(state, action)))
        return best

    if Player(state) == MIN:
        best = +infinity
        for each action in Actions(state):
            best = min(best, MINIMAX(Result(state, action)))
        return best
```

Property	Value
Complete	Yes, for finite game trees
Optimal	Yes, against an opponent who also plays optimally
Time Complexity	$O(b^m)$ — b = branching factor, m = depth of the tree
Space Complexity	$O(bm)$ — explores depth-first

ADVANTAGES

- Guarantees the optimal move against a perfectly rational opponent
- Simple, elegant recursive formulation
- Forms the theoretical foundation of all game-playing AI

LIMITATIONS

- Extremely expensive for games with large branching factor/depth (e.g., chess)
- Requires exploring the entire game tree in its basic form
- Impractical without pruning or depth-limiting in real games

QUICK REVISION — MINIMAX

- MAX picks maximum of children's values; MIN picks minimum.
- Values computed bottom-up, from terminal states to root.
- The root's chosen child gives the optimal first move.
- MINIMAX is complete and optimal but computationally expensive: $O(b^m)$.
- Motivates the need for Alpha-Beta Pruning (next topic) to reduce this cost.

Alpha-Beta Pruning

WHY ALPHA-BETA PRUNING IS NEEDED

MINIMAX must explore the **entire** game tree to compute the optimal move, which is extremely expensive for real games.

Alpha-Beta Pruning is an optimisation of MINIMAX that **skips (prunes) branches** that cannot possibly influence the final decision — giving the exact same result as MINIMAX, but much faster.

Term	Meaning
Alpha (α)	The best value that MAX can guarantee so far along the current path (initially $-\infty$)
Beta (β)	The best value that MIN can guarantee so far along the current path (initially $+\infty$)
Pruning / Cut-off	Stopping the exploration of a branch early because it cannot change the final decision

THE PRUNING RULE

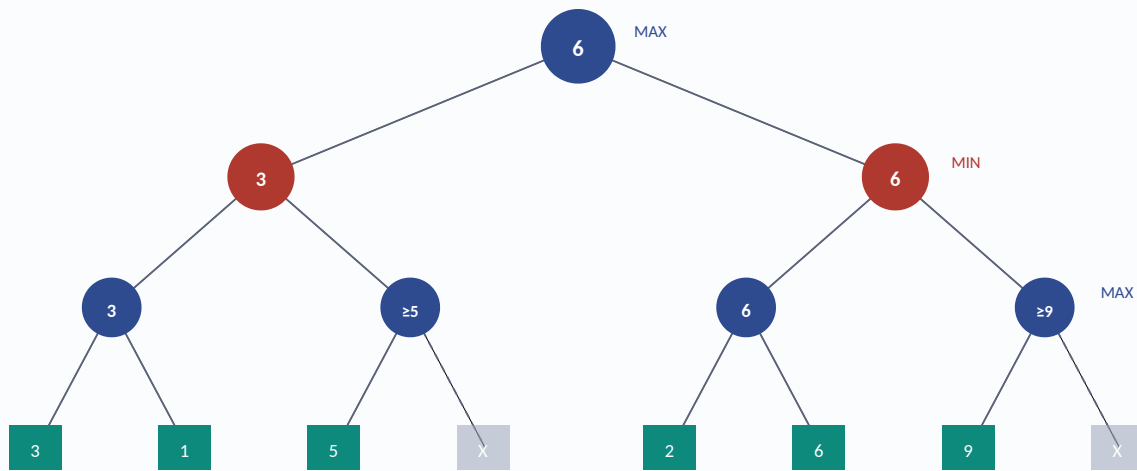
If $\beta \leq \alpha$ at any node $\rightarrow$ PRUNE remaining children

A MAX node prunes remaining children once its current best value $\geq \beta$ (inherited from an ancestor MIN node). A MIN node prunes remaining children once its current best value $\leq \alpha$ (inherited from an ancestor MAX node).

- **How Alpha is updated:** at a MAX node, α is updated to the highest value found among children examined so far.
- **How Beta is updated:** at a MIN node, β is updated to the lowest value found among children examined so far.
- **When pruning occurs:** as soon as a node's value would make its parent's decision worse than an already-guaranteed alternative elsewhere in the tree, further exploration of that node's remaining children is skipped.
- **Why the final answer is unchanged:** pruned branches are proven — by the alpha/beta bounds — to be branches a rational opponent would never allow to be reached, so ignoring them cannot change the true MINIMAX value at the root.

7.1 Step-by-Step Example with Pruning

We apply Alpha-Beta Pruning to the **exact same tree as Fig 6.1** (leaves, left to right: 3, 1, 5, 4, 2, 6, 9, 0). The final value returned must therefore match MINIMAX's answer of **6** — Alpha-Beta only skips evaluating some leaves; it never changes the result.



Grey "X" leaves (values 4 and 0) are pruned — never evaluated; every other node matches Fig 6.1 exactly

Fig 7.1 — Same tree as Fig 6.1: Alpha-Beta reaches the identical final value (6) while skipping 2 of the 8 leaves

Trace of Alpha-Beta values (Fig 7.1, left to right, depth-first)

Step	Node visited	α	β	Action
1	Leaves 3, 1 under left MAX node	$-\infty$	$+\infty$	Left MAX node = $\max(3, 1) = 3$
2	Node B (MIN)	$-\infty$	3	β updated to 3 after first child returns 3
3	Leaf 5 under 2nd MAX node	$-\infty$	3	Node's running value = 5; since $5 \geq \beta(3) \rightarrow$ PRUNE remaining leaf (4)
4	Node B (MIN) finalised	$-\infty$	3	$B = \min(3, \geq 5) = 3$ (2nd child's exact value irrelevant)
5	Root A (MAX)	3	$+\infty$	α updated to 3 (B's value)
6	Leaves 2, 6 under 3rd MAX node (C1)	3	$+\infty$	Both evaluated (no cutoff yet); $C1 = \max(2, 6) = 6$
7	Node C (MIN)	3	6	β updated to 6 after first child (C1) returns 6; $6 > \alpha(3)$, so no prune here
8	Leaf 9 under 4th MAX node (C2)	3	6	Node's running value = 9; since $9 \geq \beta(6) \rightarrow$ PRUNE remaining leaf (0)
9	Node C (MIN) finalised	3	6	$C = \min(6, \geq 9) = 6$
10	Root A (MAX) finalised	6	—	$\max(3, 6) = 6 \rightarrow$ Final answer = 6 , matches MINIMAX exactly — move toward C

Only **2 of the 8 leaves** (values 4 and 0) were skipped, yet the final answer is provably identical to full MINIMAX. The pruning at Step 3 happens because that node's value (5) already guarantees B will not exceed 3 (since MIN, at B, will choose the smaller of the two); its second leaf cannot change B's value, so it is never evaluated. The same logic applies at Step 8 for C.

PSEUDOCODE — ALPHA-BETA PRUNING

```
function ALPHA_BETA(state, alpha, beta):
    if Terminal_Test(state): return Utility(state)

    if Player(state) == MAX:
        value = -infinity
        for each action in Actions(state):
            value = max(value, ALPHA_BETA(Result(state, action), alpha, beta))
            alpha = max(alpha, value)
            if alpha >= beta:
                break                # BETA CUT-OFF: prune remaining children
        return value

    if Player(state) == MIN:
        value = +infinity
        for each action in Actions(state):
            value = min(value, ALPHA_BETA(Result(state, action), alpha, beta))
            beta = min(beta, value)
            if beta <= alpha:
                break                # ALPHA CUT-OFF: prune remaining children
        return value

# Initial call: ALPHA_BETA(root, -infinity, +infinity)
```

7.2 Comparison — MINIMAX vs Alpha-Beta Pruning

Point	MINIMAX	Alpha-Beta Pruning
Nodes explored	All nodes in the tree	Only nodes that can affect the final decision
Final answer	Optimal move	Exactly the same optimal move
Time Complexity	$O(b^m)$	$O(b^{m/2})$ with optimal move ordering
Extra bookkeeping	None	Maintains α and β at every node
Practical use	Only feasible for very small games	Used in almost all real game-playing programs

EFFECT OF MOVE ORDERING

Alpha-Beta Pruning's efficiency depends heavily on the order moves are examined:

- **Good move ordering** (best moves examined first) — produces the maximum possible pruning, reducing time complexity to roughly $O(b^{m/2})$, effectively doubling the search depth achievable in the same time as plain MINIMAX.
- **Bad move ordering** (worst moves examined first) — little or no pruning occurs, and performance falls back toward plain MINIMAX's $O(b^m)$.

QUICK REVISION — ALPHA-BETA PRUNING

- Alpha-Beta Pruning gives the exact same result as MINIMAX, but explores far fewer nodes.
- α = best value MAX can guarantee so far; β = best value MIN can guarantee so far.
- Pruning rule: cut off remaining children when $\beta \leq \alpha$.
- Best case time complexity: $O(b^{m/2})$ with good move ordering.
- Move ordering is critical — good ordering roughly doubles the practical search depth.

Unit C — Quick Revision Sheet

1. Important Definitions

Term	Definition
Search	Systematically exploring a state space to find a path from an initial state to a goal state
State Space	The set of all states reachable from the initial state
Frontier	The set of generated but not-yet-expanded nodes
Uninformed Search	Search with no knowledge of goal distance, using only the problem definition
Informed Search	Search guided by a heuristic function estimating distance to the goal
Heuristic Function $h(n)$	An estimate of the cost from node n to the goal
Admissible Heuristic	A heuristic that never overestimates the true cost to the goal
Local Search	Search that tracks only a current state and moves to better neighbours, without keeping paths
Local Maximum	A state better than all its neighbours but not the best overall (global) state
Adversarial Search	Search used in competitive multi-agent settings, e.g. two-player games
Utility Value	A numeric score assigned to a terminal game state showing its value to MAX
Alpha (α)	The best value MAX can guarantee so far along the current path
Beta (β)	The best value MIN can guarantee so far along the current path

2. Important Algorithms — One-Line Summary

Algorithm	One-line idea
BFS	Explore level-by-level using a FIFO queue
DFS	Explore as deep as possible using a LIFO stack
Depth-Limited Search	DFS with a fixed maximum depth L
Iterative Deepening Search	Repeated DLS with gradually increasing L
Uniform Cost Search	Expand the lowest cumulative-cost node first (priority queue)
Greedy Best-First Search	Expand the node with lowest $h(n)$ only
A* Search	Expand the node with lowest $f(n) = g(n) + h(n)$
Hill Climbing	Always move to a better neighbouring state; stop at a peak
Simulated Annealing	Hill Climbing + occasional random worse moves, controlled by temperature
MINIMAX	MAX maximises, MIN minimises, values propagate bottom-up through the game tree
Alpha-Beta Pruning	MINIMAX + skip branches proven irrelevant to the final decision

3. Important Differences (Master Comparison Tables)

BFS vs DFS

Point	BFS	DFS
Structure	Queue	Stack
Complete	Yes	No
Optimal	Yes (equal costs)	No
Space	$O(b^d)$ — high	$O(bm)$ — low

Uninformed vs Informed Search

Point	Uninformed	Informed
Knowledge used	Problem definition only	Problem definition + heuristic
Efficiency	Lower	Higher (with good heuristic)

Greedy Best-First vs A*

Point	Greedy Best-First	A*
$f(n)$	$h(n)$	$g(n) + h(n)$
Optimal	No	Yes (admissible h)

Hill Climbing vs Simulated Annealing

Point	Hill Climbing	Simulated Annealing
Accepts worse moves?	No	Yes, probabilistically
Escapes local maxima?	No	Yes

MINIMAX vs Alpha-Beta Pruning

Point	MINIMAX	Alpha-Beta
Nodes explored	All	Only relevant ones
Time Complexity	$O(b^m)$	$O(b^{m/2})$ best case

4. Important Formulas

A* SEARCH

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual cost from start to n | $h(n)$ = estimated cost from n to goal | $f(n)$ = estimated total path cost through n

SIMULATED ANNEALING ACCEPTANCE PROBABILITY

$$P(\text{accept worse move}) = e^{(\Delta E / T)}$$

ΔE = change in value (negative for a worse move) | T = current temperature

MINIMAX VALUE

MAX node: $\max(\text{children})$

MIN node: $\min(\text{children})$

ALPHA-BETA PRUNING CONDITION

Prune when $\beta \leq \alpha$

5. Practice Questions

1. Define searching in Artificial Intelligence and list the components of a search problem.
2. Differentiate between a search tree and a search graph.
3. Explain Breadth First Search with a suitable example. State its completeness and optimality.
4. Explain Depth First Search and discuss why it is not optimal.
5. What is Depth-Limited Search? How does Iterative Deepening Search overcome its main drawback?
6. Explain Uniform Cost Search with a numerical example showing how it selects the optimal path.
7. Differentiate between uninformed and informed search strategies.
8. What is a heuristic function? Explain the concept of an admissible heuristic.
9. Explain the A* search algorithm. Given $g(n)$ and $h(n)$ values for a small tree, calculate $f(n)$ for each node and determine the path A* would choose.
10. Explain Hill Climbing and describe the problems of local maximum, plateau, and ridge with diagrams.
11. Explain Simulated Annealing and how the temperature parameter helps escape local maxima.
12. What is adversarial search? Explain the roles of MAX and MIN players with an example.
13. Explain the MINIMAX algorithm with a step-by-step numerical example on a game tree.
14. What is Alpha-Beta Pruning? Explain the meaning of alpha and beta with the pruning condition.
15. Using a game tree, show a complete example of Alpha-Beta Pruning, clearly marking the pruned branches and explaining why the final answer matches MINIMAX.

FINAL EXAM TIP

For every long-answer question in this unit, follow the structure: *Introduction* → *Definition* → *Working/Algorithm* → *Diagram* → *Worked Example* → *Advantages* → *Limitations* → *Conclusion*. This structure alone covers most of what examiners look for in Unit C answers.